Status	Finished
Started	Monday, 28 September 2026, 9:08 AM
Completed	Monday, 28 September 2026, 9:18 AM
Duration	9 mins 35 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

The **DNA sequence** is composed of a series of nucleotides abbreviated as `'A'`, `'C'`, `'G'`, and `'T'`.

- For example, `"ACGAATTCCG"` is a **DNA sequence**.

When studying **DNA**, it is useful to identify repeated sequences within the DNA.

Given a string `s` that represents a **DNA sequence**, return all the `10`**-letter-long** sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in **any order**.

Example 1:

```
Input: s = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT"
Output: ["AAAAACCCCC","CCCCCAAAAA"]
```

Example 2:

```
Input: s = "AAAAAAAAAAAAA"
Output: ["AAAAAAAAAA"]
```

For example:

Input	Result
AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC CCCCCAAAAA

Answer: (penalty regime: 0 %)

```python
import sys

def findRepeatedDnaSequences(s: str):
    seen = set()
    repeated = set()

    # Slide a window of size 10 across the string
    for i in range(len(s) - 9):
        ten_letter_seq = s[i:i+10]
        if ten_letter_seq in seen:
            repeated.add(ten_letter_seq)
        else:
            seen.add(ten_letter_seq)

    # Print each repeated sequence on a new line to match the output requirement
    for seq in repeated:
        print(seq)

def main():
    # Read the full string from standard input to handle automated system evaluation
    input_data = sys.stdin.read().splitlines()
    for line in input_data:
        cleaned = line.strip()
        if cleaned:
            findRepeatedDnaSequences(cleaned)

if __name__ == "__main__":
    main()

```

	Input	Expected	Got	
✓	AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCCCCCCCAAAAA	AAAAACCCCCCCCCCAAAAA	✓
✓	AAAAAAAAAAAAA	AAAAAAAAAA	AAAAAAAAAA	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Program to print all the distinct elements in an array. Distinct elements are nothing but the unique (non-duplicate) elements present in the given array.

Input Format:

First line take an Integer input from stdin which is array length n.

Second line take n Integers which is inputs of array.

Output Format:

Print the Distinct Elements in Array in single line which is space Separated

Example Input:

5

1 2 2 3 4

Output:

1 2 3 4

Example Input:

6

1 1 2 2 3 3

Output:

1 2 3

For example:

Input	Result
5 1 2 2 3 4	1 2 3 4

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all tokens from standard input to accommodate any newline/space layout
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # The first token is the size of the array 'n'
    n = int(input_data[0])

    # Extract the next n elements from the token list
    elements = input_data[1:1 + n]

    # Use a dictionary to remove duplicates while preserving the exact order of appearance
    distinct_elements = list(dict.fromkeys(elements))

    # Print the unique elements on a single line separated by a space
    print(" ".join(distinct_elements) + " ")

if __name__ == "__main__":
    main()

```

	Input	Expected	Got	
✓	5 1 2 2 3 4	1 2 3 4	1 2 3 4	✓
✓	6 1 1 2 2 3 3	1 2 3	1 2 3	✓
✓	5 11 22 11 22 11	11 22	11 22	✓
✓	10 1 2 3 4 5 1 2 3 4 5	1 2 3 4 5	1 2 3 4 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given a tuple and a positive integer k, the task is to find the count of distinct pairs in the tuple whose sum is equal to **K**.

Examples:

Input: t = (5, 6, 5, 7, 7, 8), K = 13
Output: 2
Explanation:
Pairs with sum K(= 13) are {(5, 8), (6, 7), (6, 7)}.
Therefore, distinct pairs with sum K(= 13) are { (5, 8), (6, 7) }.
Therefore, the required output is 2.

For example:

Input	Result
1,2,1,2,5 3	1
1,2 0	0

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all lines from standard input
    lines = sys.stdin.read().splitlines()
    if not lines:
        return

    idx = 0
    while idx < len(lines):
        line1 = lines[idx].strip()
        if not line1:
            idx += 1
            continue

        if idx + 1 >= len(lines):
            break

        line2 = lines[idx+1].strip()

        # Dynamically split by comma or whitespace depending on input format
        if ',' in line1:
            parts = line1.split(',')
        else:
            parts = line1.split()

        # Safely convert tuple elements to integers
        t = []
        for p in parts:
            try:
                t.append(int(p.strip()))
            except ValueError:
                pass

        # Safely convert K to an integer
        try:
            k = int(line2)
        except ValueError:
            idx += 1
            continue

        # Count the frequency of each element in the tuple
        counts = {}
        for num in t:
            counts[num] = counts.get(num, 0) + 1
```

```
46
47          distinct_pairs = set()
48          for x in counts:
49              y = k - x
50              if x < y:
51                  if y in counts:
52                      distinct_pairs.add((x, y))
```

	Input	Expected	Got	
✓	5,6,5,7,7,8 13	2	2	✓
✓	1,2,1,2,5 3	1	1	✓
✓	1,2 0	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.There is only **one repeated number** in `nums`, return *this repeated number*. Solve the problem using set.

Example 1:

```
Input: nums = [1,3,4,2,2]
```

```
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
```

```
Output: 3
```

For example:

Input	Result
1 3 4 4 2	4

Answer: (penalty regime: 0 %)

```
import sys

def findDuplicate(nums: list) -> int:
    seen = set()
    for num in nums:
        if num in seen:
            return num
        seen.add(num)
    return -1

def main():
    # Read all tokens from standard input to handle any space or newline separation
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # Convert inputs to integers
    nums = [int(x) for x in input_data]
    print(findDuplicate(nums))

if __name__ == "__main__":
    main()

```

	Input	Expected	Got	
✓	1 3 4 4 2	4	4	✓
✓	1 2 2 3 4 5 6 7	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Coders here is a simple task for you, Given string str. Your task is to check whether it is a binary string or not by using python set.

Examples:

Input: str = "01010101010"

Output: Yes

Input: str = "REC101"

Output: No

For example:

Input	Result
01010101010	Yes
010101 10101	No

Answer: (penalty regime: 0 %)

```
import sys

def check_binary_string(s: str) -> str:
    # Use a set to extract unique characters from the string
    char_set = set(s)

    # If the set contains any characters other than '0' or '1', it's not a binary string
    if char_set.issubset({'0', '1'}) and len(char_set) > 0:
        return "Yes"
    return "No"

def main():
    # Read standard input lines to handle trailing spaces or automated testing platforms
    lines = sys.stdin.read().splitlines()
    for line in lines:
        if line:  # Skip completely empty lines, but preserve spaced string evaluation
            print(check_binary_string(line))

if __name__ == "__main__":
    main()

```

	Input	Expected	Got	
✓	01010101010	Yes	Yes	✓
✓	REC123	No	No	✓
✓	010101 10101	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Check if a set is a subset of another set.

Example:

Sample Input1:

mango apple

mango orange

mango

output1:

yes

set3 is subset of set1 and set2

```
input2:
```

```
mango orange
```

```
banana orange
```

```
grapes
```

```
output2:
```

no

For example:

Test	Input	Result
1	mango apple mango orange mango	yes set3 is subset of set1 and set2
2	mango orange banana orange grapes	No

Answer: (penalty regime: 0 %)

```
import sys

def check_subset():
    # Read all lines from standard input and filter out empty ones
    lines = [line.strip() for line in sys.stdin.read().splitlines() if line.strip()]
    if len(lines) < 3:
        return

    # Parse the three sets by splitting the words on each line
    set1 = set(lines[0].split())
    set2 = set(lines[1].split())
    set3 = set(lines[2].split())

    # Check if set3 is a subset of both set1 and set2
    if set3.issubset(set1) and set3.issubset(set2):
        print("yes")
        print("set3 is subset of set1 and set2")
    else:
        print("No")

if __name__ == "__main__":
    check_subset()

```

	Test	Input	Expected	Got	
✓	1	mango apple mango orange mango	yes set3 is subset of set1 and set2	yes set3 is subset of set1 and set2	✓
✓	2	mango orange banana orange grapes	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

You are given an integer tuple `nums` containing distinct numbers. Your task is to perform a sequence of operations on this tuple until it becomes empty. The operations are defined as follows:

1. If the first element of the tuple has the smallest value in the entire tuple, remove it.
2. Otherwise, move the first element to the end of the tuple.

You need to return an integer denoting the number of operations required to make the tuple empty.

Constraints

- The input tuple `nums` contains distinct integers.
- The operations must be performed using tuples and sets to maintain immutability and efficiency.
- Your function should accept the tuple `nums` as input and return the total number of operations as an integer.

Example:

Input: nums = (3, 4, -1)
Output: 5

Explanation:
Operation 1: [3, 4, -1] -> First element is not the smallest, move to the end -> [4, -1, 3]
Operation 2: [4, -1, 3] -> First element is not the smallest, move to the end -> [-1, 3, 4]
Operation 3: [-1, 3, 4] -> First element is the smallest, remove it -> [3, 4]
Operation 4: [3, 4] -> First element is the smallest, remove it -> [4]
Operation 5: [4] -> First element is the smallest, remove it -> []
Total operations: 5

For example:

Test	Result
`print(count_operations((3, 4, -1)))`	5

Answer: (penalty regime: 0 %)

Reset answer

```
def count_operations(nums: tuple) -> int:
    # Handle the base case where the input tuple is already empty
    if not nums:
        return 0

    # Sort the elements to know the sequence of expected minimums in O(N log N)
    sorted_elements = sorted(nums)

    # A pointer tracking which minimum value we are currently searching to eliminate
    min_idx = 0
    total_ops = 0

    # Process using a loop that simulates the queue operations via structural slicing
    while min_idx < len(sorted_elements):
        current_min = sorted_elements[min_idx]
        first_element = nums[0]

        # Every cycle requires an incrementing operation check
        total_ops += 1

        if first_element == current_min:
            # If it matches the absolute smallest element, slice off the first position
            nums = nums[1:]
            min_idx += 1
        else:
            # Shift the first item to the back of the tuple sequence
            nums = nums[1:] + (first_element,)

    return total_ops

```

	Test	Expected	Got	
✓	`print(count_operations((3, 4, -1)))`	5	5	✓
✓	`print(count_operations((1, 2, 3, 4, 5)))`	5	5	✓
✓	`print(count_operations((5, 4, 3, 2, 1)))`	15	15	✓
✓	`print(count_operations((42, )))`	1	1	✓
✓	`print(count_operations((-2, 3, -5, 4, 1)))`	11	11	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

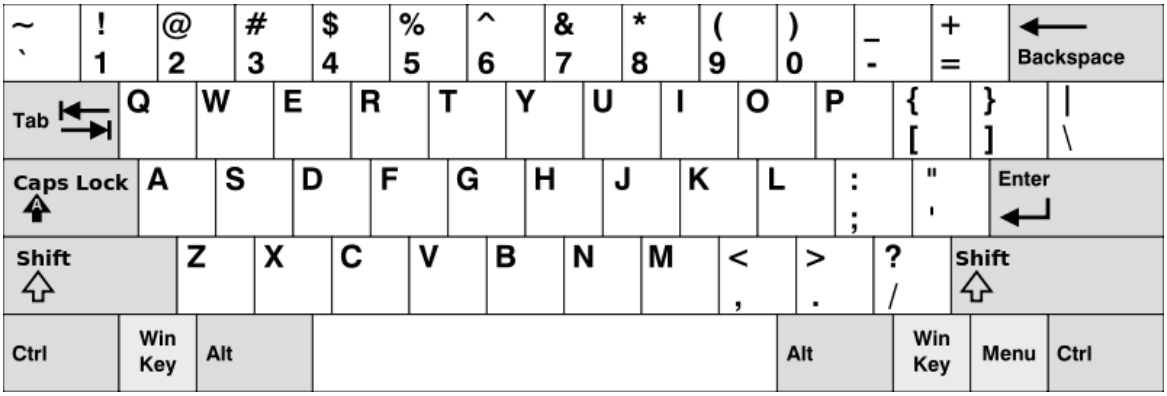
Correct

Mark 1.00 out of 1.00

Given an array of strings `words`, return *the words that can be typed using letters of the alphabet on only one row of American keyboard like the image below.*

In the **American keyboard**:

- the first row consists of the characters `"qwertyuiop"`,
- the second row consists of the characters `"asdfghjkl"`, and
- the third row consists of the characters `"zxcvbnm"`.



Example 1:

```
Input: words = ["Hello","Alaska","Dad","Peace"]
Output: ["Alaska","Dad"]
```

Example 2:

```
Input: words = ["omk"]
Output: []
```

Example 3:

```
Input: words = ["adsdf","sfd"]
Output: ["adsdf","sfd"]
```

For example:

Input	Result
4 Hello Alaska Dad Peace	Alaska Dad
2 adsfd afd	adsfd afd

Answer: (penalty regime: 0 %)

```
n = int(input())

rows = [
    set("qwertyuiop"),
    set("asdfghjkl"),
    set("zxcvbnm")
]

words = [input().strip() for _ in range(n)]
result = []

for word in words:
```

```
    w = word.lower()

    if any(all(ch in row for ch in w) for row in rows):
        result.append(word)

if result:
    for word in result:
        print(word)
else:
    print("No words")

```

	Input	Expected	Got	
✓	4 Hello Alaska Dad Peace	Alaska Dad	Alaska Dad	✓
✓	1 omk	No words	No words	✓
✓	2 adsfd afd	adsfd afd	adsfd afd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Write a program to eliminate the common elements in the given 2 arrays and print only the non-repeating elements and the total number of such non-repeating elements.

Input Format:

The first line contains space-separated values, denoting the size of the two arrays in integer format respectively.

The next two lines contain the space-separated integer arrays to be compared.

Sample Input:

5 4

1 2 8 6 5

2 6 8 10

Sample Output:

1 5 10

3

Sample Input:

5 5

1 2 3 4 5

1 2 3 4 5

Sample Output:

NO SUCH ELEMENTS

For example:

Input	Result
5 4 1 2 8 6 5 2 6 8 10	1 5 10 3
5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS

Answer: (penalty regime: 0 %)

```
n, m = map(int, input().split())

a = list(map(int, input().split()))
b = list(map(int, input().split()))

# Elements that occur in only one of the two arrays
result = []

for x in a:
    if x not in b and x not in result:
        result.append(x)

for x in b:
    if x not in a and x not in result:
        result.append(x)

if result:
    print(*result)
    print(len(result))
else:
    print("NO SUCH ELEMENTS")

```

	Input	Expected	Got	
✓	5 4 1 2 8 6 5 2 6 8 10	1 5 10 3	1 5 10 3	✓
✓	3 3 10 10 10 10 11 12	11 12 2	11 12 2	✓
✓	5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS	NO SUCH ELEMENTS	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

There is a malfunctioning keyboard where some letter keys do not work. All other keys on the keyboard work properly.

Given a string text of words separated by a single space (no leading or trailing spaces) and a string brokenLetters of all distinct letter keys that are broken, return the number of words in text you can fully type using this keyboard.

Example 1:

Input: text = "hello world", brokenLetters = "ad"

Output:

1

Explanation: We cannot type "world" because the 'd' key is broken.

For example:

Input	Result
hello world ad	1
Faculty Upskilling in Python Programming ak	2

Answer: (penalty regime: 0 %)

```
text = input().strip().lower()
brokenLetters = input().strip().lower()

words = text.split()
count = 0

for word in words:
    if all(ch not in brokenLetters for ch in word):
        count += 1

print(count)
```

	Input	Expected	Got	
✓	hello world ad	1	1	✓
✓	Welcome to REC e	1	1	✓
✓	Faculty Upskilling in Python Programming ak	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.